

The Concurrent Programming Abstraction

Contiene illustrazioni prese da:

M. Ben-Ari. Principles of Concurrent and Distributed Programming, Second edition © M. Ben-Ari 2006

Programmazione concorrente

- ▶ Un **programma concorrente** contiene **due o più processi** (o sottoprocessi - **threads**) che **lavorano assieme** per eseguire una determinata applicazione
- ▶ Ciascun (sotto)processo è un **programma sequenziale**
- ▶ I (sotto)processi **comunicano tra loro** utilizzando variabili condivise (**shared memory**) oppure scambiandosi messaggi (**message passing**)
- ▶ **ASTRAZIONE:** i (sotto)processi sono in **esecuzione contemporanea**
 - ▶ È un'astrazione... in realtà, potrebbe non essere esattamente così...

Le origini

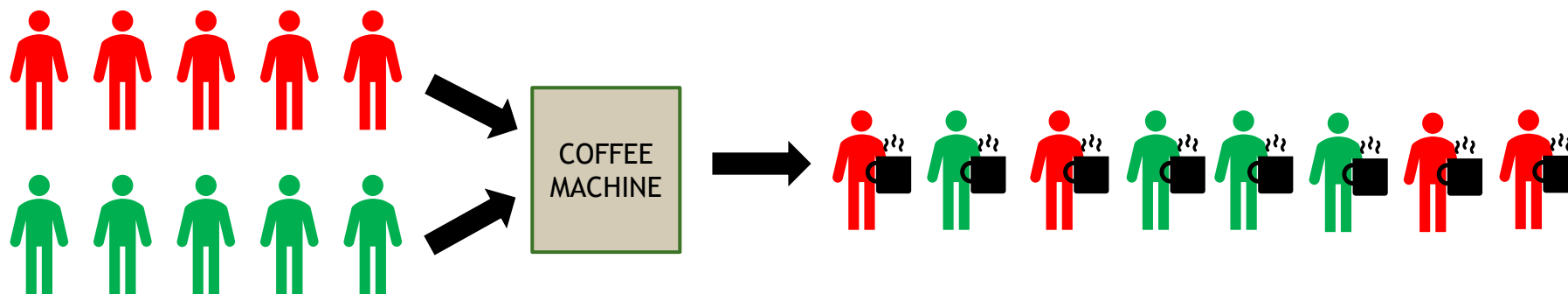
- ▶ La programmazione nasce negli anni '60 nell'ambito dei **sistemi operativi**
- ▶ Nasce dall'esigenza di far **continuare l'esecuzione dei programmi** durante lo svolgimento di (lunghe) **operazioni di I/O**
- ▶ Negli anni '80, si diffondono sistemi operativi con **preemptive multitasking**
 - ▶ Possibilità di mantenere attivi più programmi (o processi) contemporaneamente **alternandone l'esecuzione** nel processore (**interleaving**)
 - ▶ L'alternarsi dei programmi è sotto il controllo del sistema operativo che assegna in modo equo il tempo di CPU ai processi attivi
 - ▶ Richiede supporto hardware (**interrupt** programmabili)

Sistemi multiprocessore

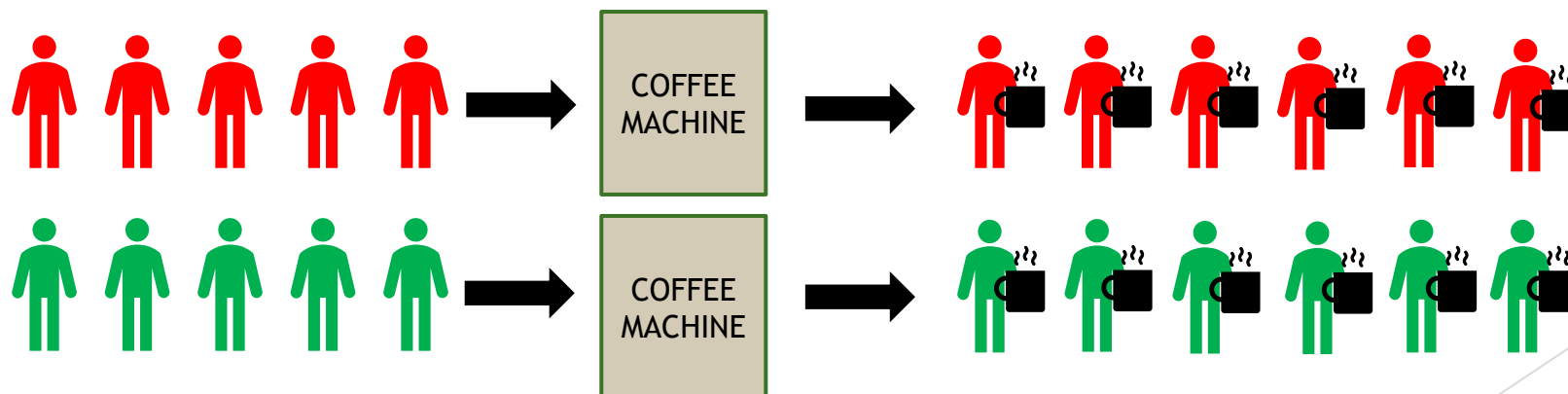
- ▶ Successivamente furono introdotti i **sistemi multiprocessore**, dotati di più CPU (e/o di CPU multi-core)
- ▶ I sistemi multiprocessore:
 - ▶ consentono di eseguire diversi **processi in parallelo** (ossia, contemporaneamente su CPU diverse)
 - ▶ consentono di eseguire le applicazioni (costituite da più processi) **più velocemente** rispetto all'esecuzione su singola CPU

Concorrenza VS Parallelismo

- Programmazione **concorrente**: 1 CPU e N task contemporaneamente



- Programmazione **parallela**: N CPU e N task contemporaneamente



Ma frequentemente
capita di avere N CPU
e M task con $N < M$

Esecuzione non sequenziale

- ▶ Concorrenza e parallelismo sono accomunati da un aspetto:
 - ▶ **Esecuzione non sequenziale** del programma
 - ▶ Idealmente, **ogni (sotto)processo** che costituisce il programma **ha un proprio program counter** che avanza autonomamente
- ▶ Esempio con due processi, ognuno con due comandi:
 - ▶ **Processo P:** p1,p2
 - ▶ **Processo Q:** q1,q2

Possibili **interleaving** in caso di **concorrenza (singola CPU)**:

p1 -> p2 -> q1 -> q2

p1 -> q1 -> p2 -> q2

p1 -> q1 -> q2 -> p2

q1 -> p1 -> p2 -> q2

q1 -> p1 -> q2 -< p2

q1 -> q2 -> p1 -> p2

Esempio

Algorithm 2.1: Trivial concurrent program	
integer $n \leftarrow 0$	
p	q
integer $k1 \leftarrow 1$ p1: $n \leftarrow k1$	integer $k2 \leftarrow 2$ q1: $n \leftarrow k2$

- ▶ n è una variabile globale (inizializzata a 0)
- ▶ $k1$ e $k2$ sono variabili locali
- ▶ p e q sono processi concorrenti

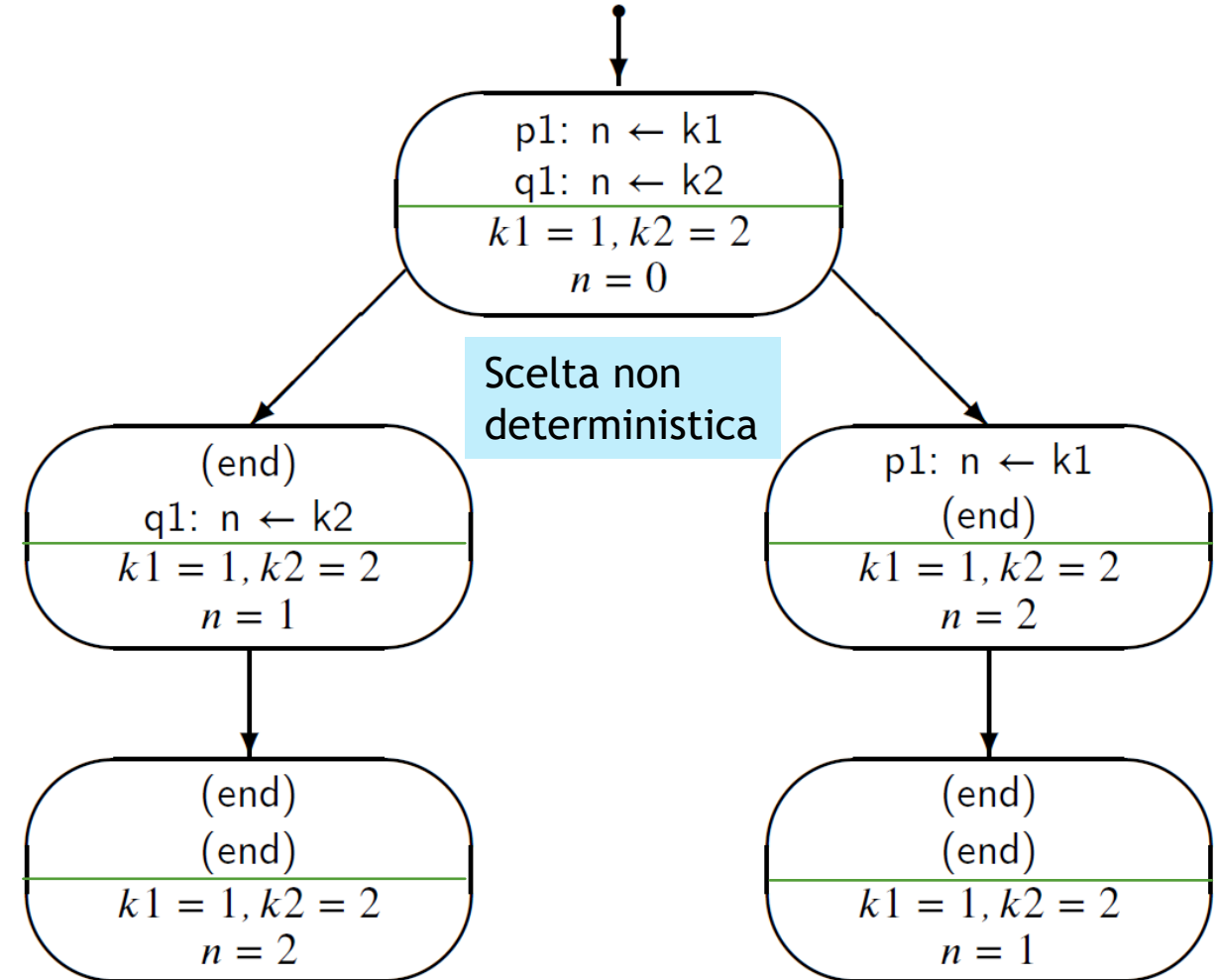
qual è il valore di n al termine?

Analizziamo i possibili comportamenti dell'esempio tramite un **sistema di transizioni** rappresentato con un grafo

- È dato dalla semantica del linguaggio
- Descrive tutti i possibili passi di computazione che il programma può fare
- Tiene conto dei possibili interleaving dei processi

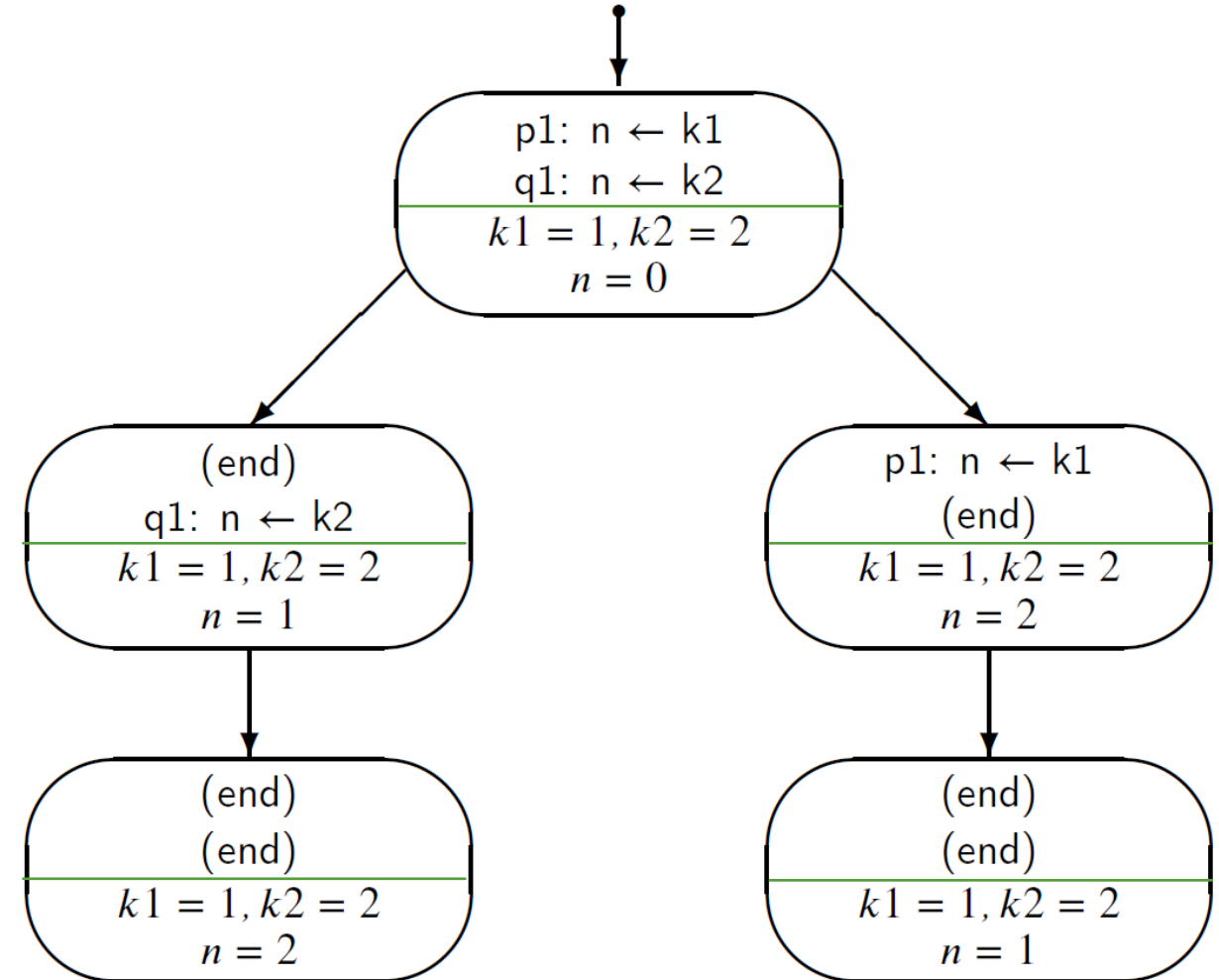
Esempio:

- n ha due possibili valori finali: 2 e 1...



Il sistema di transizioni:

- ▶ È **non deterministico** (lo stato iniziale ha due alternative possibili)
- ▶ Il non determinismo **ASTRAE** dal **criterio usato dallo scheduler** del sistema operativo per scegliere quale processo far avanzare (ad es. priorità di alcuni processi su altri)
- ▶ Conoscendo lo scheduler si potrebbe determinare che alcuni cammini nel sistema di transizioni non sono effettivamente realizzabili
- ▶ Il sistema di transizioni non fa ipotesi sullo scheduler, limitandosi a rappresentare tutte le esecuzioni teoricamente possibili



Operazioni atomiche

Algorithm 2.1: Trivial concurrent program	
integer $n \leftarrow 0$	
p	q
integer $k1 \leftarrow 1$ p1: $n \leftarrow k1$	integer $k2 \leftarrow 2$ q1: $n \leftarrow k2$

L'analisi che abbiamo fatto è corretta sotto l'**ipotesi** che le operazioni di assegnamento p1 e q1 siano **atomiche** ovvero

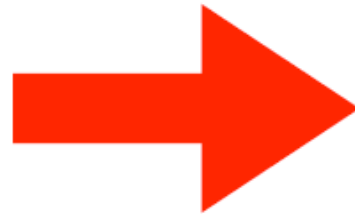
► **elementari** e **non interrompibili**

Ma non abbiamo tenuto conto del compilatore!

Il compilatore

Il compilatore traduce un **singolo comando** del linguaggio di alto livello in una **sequenza di operazioni** nel linguaggio macchina

`n = k + 1;`



`load R1,k`
`add R1,#1`
`store R1,n`

nota: R1 è
un registro

Analisi al livello macchina

In pratica quindi, **l'interleaving ha luogo al livello del linguaggio macchina**, e non del linguaggio di alto livello

- Sono le istruzioni assembler ad alternarsi, e non i comandi del linguaggio di alto livello

Analisi al livello macchina

Dovremmo rivedere l'analisi dell'esempio di programma concorrente considerandolo così:

Algorithm 2.1: Trivial concurrent program	
integer n $\leftarrow$ 0	
p	q
integer k1 $\leftarrow$ 1	integer k2 $\leftarrow$ 2
p1: n $\leftarrow$ k1	q1: n $\leftarrow$ k2
p1: load R1,k1 p2: store R1,n	q1: load R1,k2 q2: store R1,n

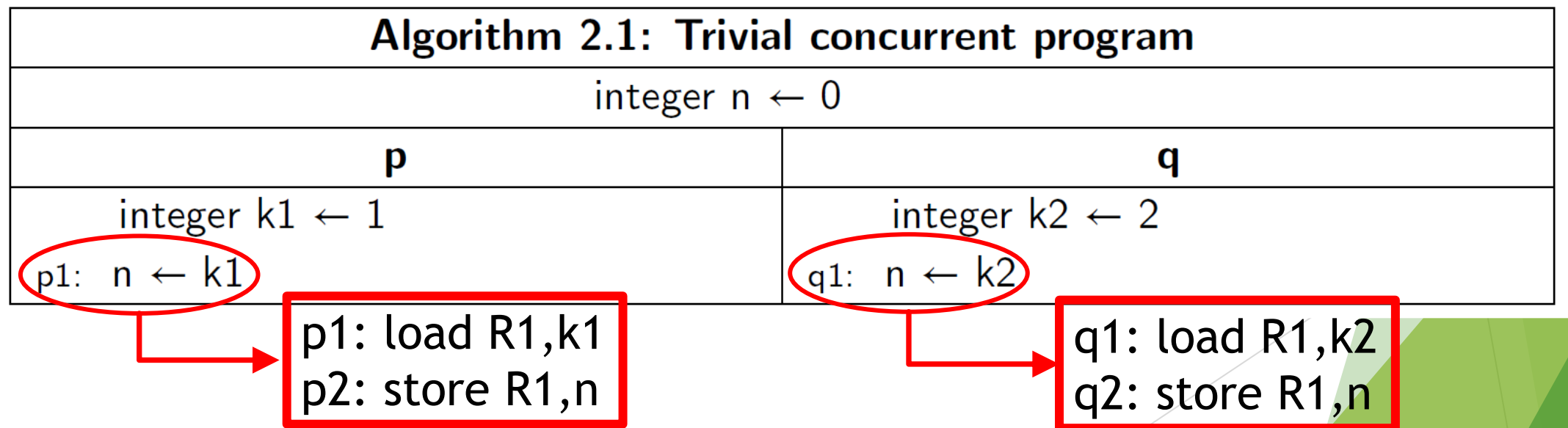
Immaginiamo che ogni processo abbia un proprio registro R1 (vedrete a S.O. come questo viene reso possibile)

TUTTO OK! Come prima, deduciamo che n alla fine potrebbe avere valore 1 o 2 (a seconda che l'ultima operazione eseguita sia p2 o q2)

E in caso di parallelismo?

Altro problema: se p e q fossero eseguiti su **due CPU diverse**, le operazioni **p2** e **q2** potrebbero essere eseguite contemporaneamente

► Non in interleaving, ma in **"true parallelism"**

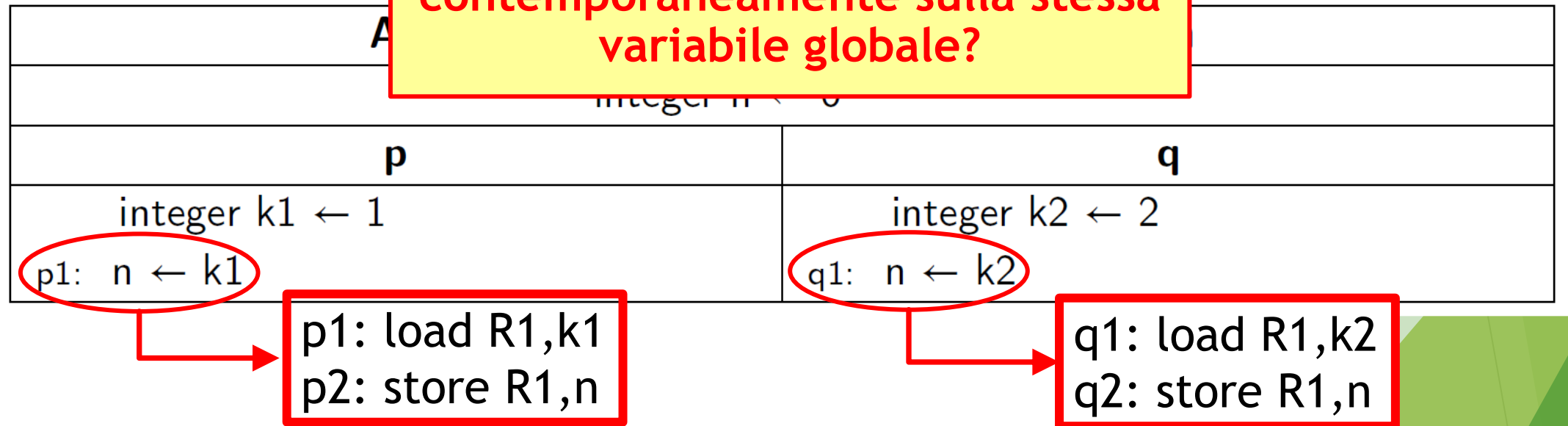


E in caso di parallelismo?

Altro problema: se p e q fossero eseguiti su **due CPU diverse**, le operazioni **p2** e **q2** potrebbero essere eseguite contemporaneamente

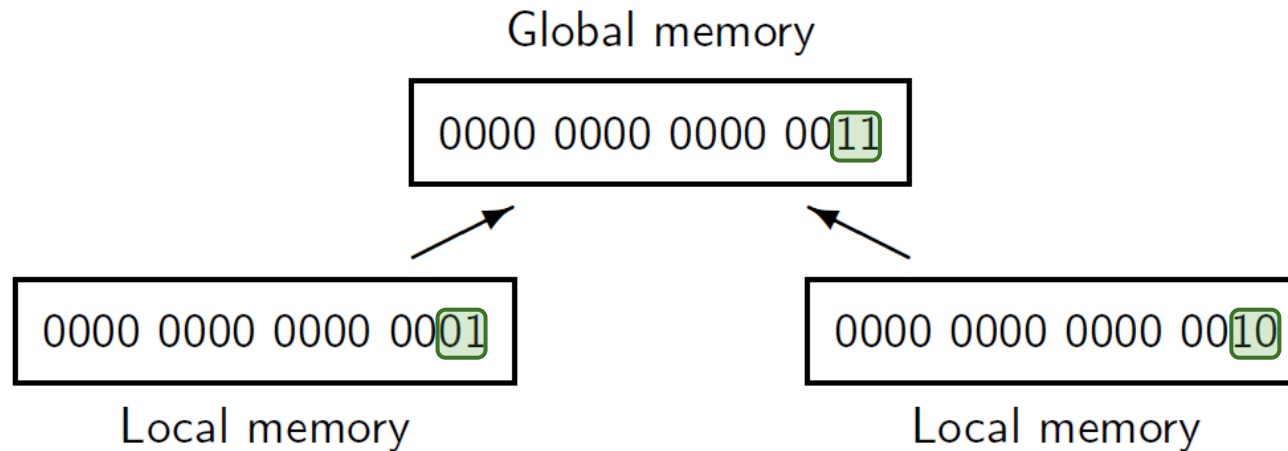
► Non in interle

Che succede se due processi cercano di scrivere contemporaneamente sulla stessa variabile globale?



Scenario possibile?

Se i due processi modificassero contemporaneamente la stessa locazione di memoria potrebbero interferire tra loro, sovrapponendo i rispettivi risultati?



Scenario possibile?

Se i due processi modificassero contemporaneamente la stessa locazione di memoria potrebbero interferire tra loro, sovrapponendo i rispettivi risultati?

NO!

Anche in caso di parallelismo **l'hardware garantisce** che **al più un processo per volta** possa scrivere su una certa locazione di memoria

Quindi...

Il fatto che anche in un contesto di **parallelismo** le modifiche contemporanee alla stessa locazione di memoria siano gestite dall'hardware eseguendole una per volta, ci riporta a uno scenario di **concorrenza**

- ▶ I processi possono operare in parallelo solo quando agiscono su locazioni di memoria distinte
- ▶ La concorrenza (interleaving) è un'astrazione utile per molte analisi dei programmi che rimangono valide anche in situazioni di parallelismo
- ▶ Come per lo scheduling del sistema operativo, anche le scelte operate dall'hardware saranno modellate tramite **non determinismo** nei sistemi di transizione

Analisi a livello di macchina VS Analisi ad alto livello

Torniamo sulla questione dell'analisi a livello macchina con un nuovo esempio:

Algorithm 2.3: Atomic assignment statements	
integer $n \leftarrow 0$	
p	q
p1: $n \leftarrow n + 1$	q1: $n \leftarrow n + 1$

Ragioniamo ad **alto livello**, quindi assumiamo che gli **assegnamenti** (del linguaggio di programmazione) siano **operazioni atomiche (indivisibili)**

Che valore ha n alla fine?

Analisi a livello di macchina VS Analisi ad alto livello

Queste sono le **due (uniche) esecuzioni possibili** del nuovo esempio di programma concorrente

- ▶ Corrispondono a due **cammini alternativi nel sistema di transizioni** che descrive il comportamento del programma
- ▶ In entrambi i casi alla fine **n vale 2**

Process p	Process q	n
p1: $n \leftarrow n+1$	q1: $n \leftarrow n+1$	0
(end)	q1: $n \leftarrow n+1$	1
(end)	(end)	2

Process p	Process q	n
p1: $n \leftarrow n+1$	q1: $n \leftarrow n+1$	0
p1: $n \leftarrow n+1$	(end)	1
(end)	(end)	2

Analisi a livello di macchina VS Analisi ad alto livello

Vediamo ora lo stesso programma a livello macchina

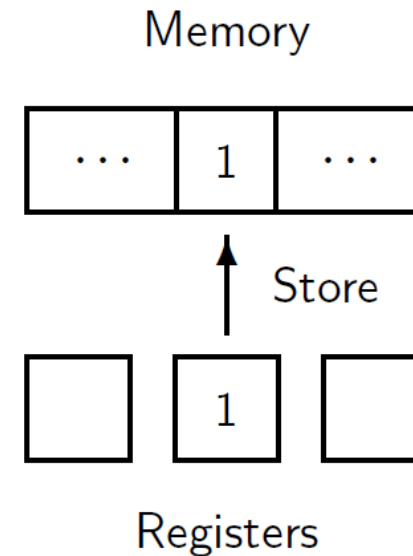
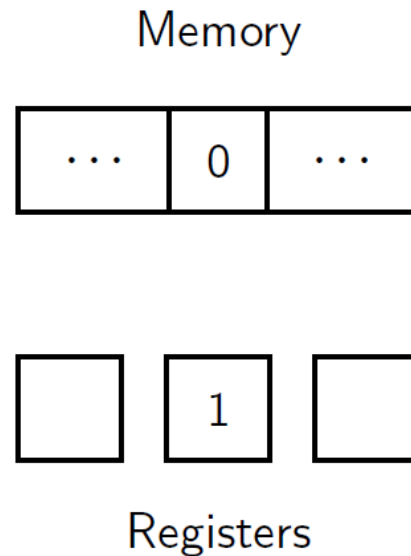
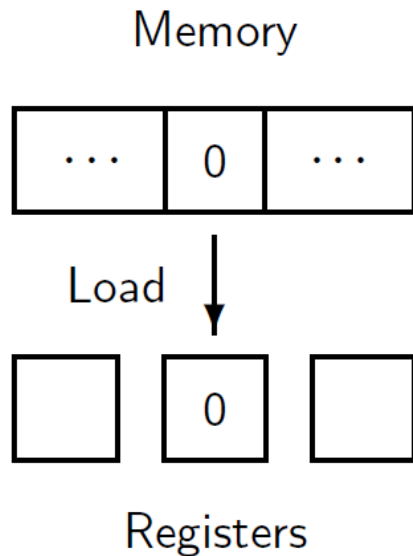
► Queste le corrispondenti istruzioni in assembler

Algorithm 2.6: Assignment statement for a register machine	
integer $n \leftarrow 0$	
p	q
p1: load R1,n p2: add R1,#1 p3: store R1,n	q1: load R1,n q2: add R1,#1 q3: store R1,n

Analisi a livello di macchina VS Analisi ad alto livello

- Rappresentazione schematica dell'esecuzione di **ognuno** dei due processi

load R1,n
add R1, #1
store R1,n



Analisi a livello di macchina VS Analisi ad alto livello

Questa è **una possibile esecuzione** descritta al livello di linguaggio macchina

- ▶ in questa particolare esecuzione **alla fine n vale 1** e non 2...
- ▶ Rispetto a prima, la load di un processo non è successiva alla store dell'altro
- ▶ chiaramente ci sono **altre esecuzioni possibili** in cui n alla fine vale 2 (ad es. p1,p2,p3,q1,q2,q3)
- ▶ **questa esecuzione non è colta dalla descrizione ad alto livello** del comportamento (la **semantica del linguaggio di programmazione**)
- ▶ l'analisi ad alto livello **non è completa** (non cattura tutti i comportamenti possibili)

Process p	Process q	n	p.R1	q.R1
p1: load R1,n	q1: load R1,n	0	?	?
p2: add R1,#1	q1: load R1,n	0	0	?
p2: add R1,#1	q2: add R1,#1	0	0	0
p3: store R1,n	q2: add R1,#1	0	1	0
p3: store R1,n	q3: store R1,n	0	1	1
(end)	q3: store R1,n	1	1	1
(end)	(end)	1	1	1

Simulare il comportamento a livello macchina nel linguaggio ad alto livello

Possibile soluzione:

- ▶ simulare l'interleaving che si ha a livello macchina con assegnamenti a **variabili temporanee (locali)** nel linguaggio ad alto livello, in modo da **separare le operazioni di load e store** sulle **variabili globali**, condivise tra processi

Algorithm 2.4: Assignment statements with one global reference	
integer n $\leftarrow$ 0	
p	q
integer temp p1: temp $\leftarrow$ n p2: n $\leftarrow$ temp + 1	integer temp q1: temp $\leftarrow$ n q2: n $\leftarrow$ temp + 1

Nota: p e q usano due variabili temp diverse

Simulare il comportamento a livello macchina nel linguaggio ad alto livello

Questo consente di catturare **ad alto livello** anche il comportamento di basso livello che nel caso precedente non veniva descritto

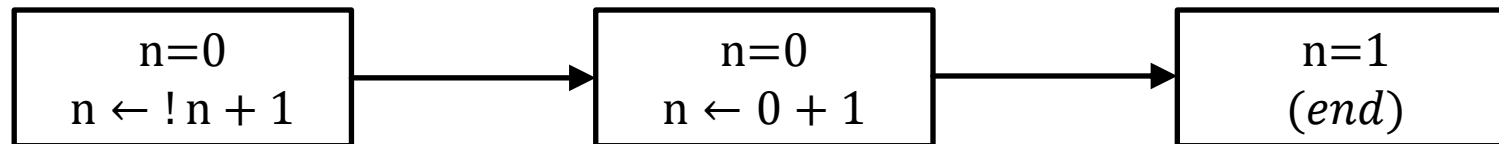
Process p	Process q	n	p.temp	q.temp
p1: temp ←n	q1: temp←n	0	?	?
p2: n←temp+1	q1: temp ←n	0	0	?
p2: n ←temp+1	q2: n←temp+1	0	0	0
(end)	q2: n ←temp+1	1	0	0
(end)	(end)	1	0	0

Nota: p e q usano
due variabili temp
diverse

Simulare il comportamento a livello macchina nel linguaggio ad alto livello

Altro modo di effettuare la stessa simulazione del comportamento di basso livello consiste nell'assumere nel linguaggio un'operazione di **dereferenziazione esplicita** delle variabili

- Invece di scrivere $n \leftarrow n + 1$ scriveremo $n \leftarrow !n + 1$ dove **!** rende esplicita l'operazione di lettura della variabile (richiede un passo di computazione)
- Esecuzione:



è una soluzione meno realistica (il valore 0 letto dalla variabile viene **temporaneamente** scritto nel codice del programma...) ma che **cattura correttamente ciò che accade a livello più basso** (lettura e scrittura in due passi)

Ricapitolando

Possiamo usare la **concorrenza come astrazione** che:

1. descrive il **comportamento** di processi tramite interleaving,
2. formalizzandoli come sistemi di transizioni non deterministici
3. ottenuti dalla semantica del linguaggio di alto livello

Inoltre, **questa astrazione** cattura

- ▶ **aspetti di parallelismo**
 - ▶ in particolare sulle variabili globali condivise
- ▶ **aspetti di interleaving a livello macchina**
 - ▶ eventualmente aggiungendo variabili temporanee locali o esplicitando le operazioni di dereferenziazione

Shared Memory VS Message Passing

Shared memory VS Message Passing

Per coordinare il lavoro del programma, è frequente che due o più (sotto)processi debbano comunicare tra loro per:

- ▶ **sincronizzarsi** in certi momenti, o
- ▶ **scambiarsi dati**

Esistono due modi diversi (e quindi modelli) per far comunicare processi:

- ▶ Memoria condivisa (**shared memory**)
- ▶ Scambio di messaggi (**message passing**)

Processi VS Thread

Caratteristiche thread

- ▶ **Condivisione della memoria:** i thread all'interno dello stesso processo condividono lo **stesso spazio di memoria** e risorse, il che facilita la comunicazione ma può portare a problemi come race condition e corruzione dei dati
- ▶ **Comunicazione:** la comunicazione tra thread è più semplice e veloce rispetto ai processi, poiché avviene **attraverso la memoria condivisa** senza bisogno di IPC
- ▶ **Gestione:** i thread sono **più leggeri** da gestire, dato che condividono le risorse del processo. Creare e distruggere thread è meno costoso in termini di risorse rispetto ai processi
- ▶ **Parallelo su CPU:** Anche i thread possono essere eseguiti su core diversi, supportando il **parallelismo all'interno di un singolo processo**
- ▶ **Crash :** Un errore in un thread può compromettere l'intero processo, inclusi gli altri thread, poiché condividono lo stesso spazio di memoria

Processi VS Thread

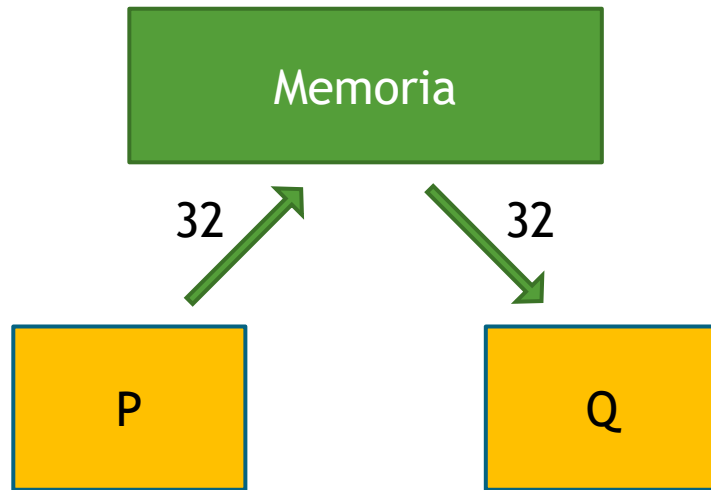
Caratteristiche processi

- ▶ **Isolamento:** Ogni processo ha il proprio **spazio di memoria indipendente**. Non condivide direttamente dati con altri processi, garantendo un maggiore isolamento e sicurezza
- ▶ **Comunicazione:** La comunicazione tra processi (Inter-Process Communication, IPC) richiede **meccanismi forniti dal sistema operativo**, come pipe, socket, o file condivisi. Questo aggiunge un certo overhead
- ▶ **Gestione:** I processi sono **più "pesanti"** da gestire in termini di risorse, poiché il sistema operativo deve allocare memoria e risorse indipendenti per ciascun processo
- ▶ **Parallelo su CPU:** Possono essere eseguiti su core diversi di una CPU, supportando il **vero parallelismo**
- ▶ **Crash:** Un errore in un processo non influenza altri processi grazie all'isolamento

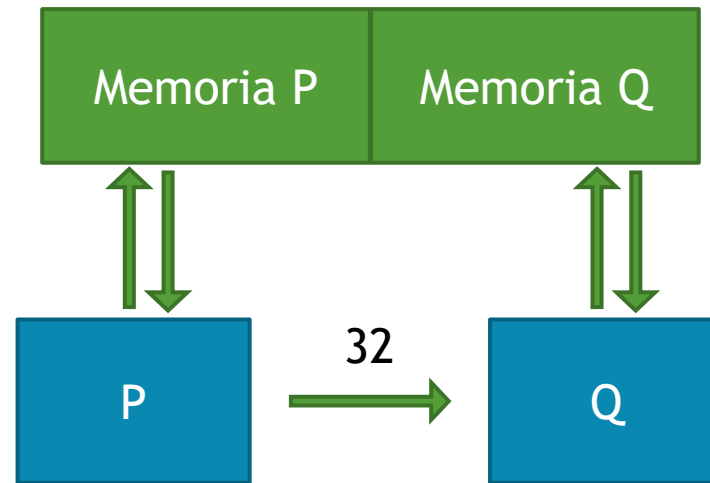
Shared Memory VS Message Passing

Shared Memory (Memoria condivisa)

- ▶ i processi (o **thread**, in questo caso) possono accedere alle stesse aree di memoria
- ▶ comunicano tra loro scrivendo e leggendo **variabili condivise**, usando gli opportuni **meccanismi di sincronizzazione**



Shared Memory

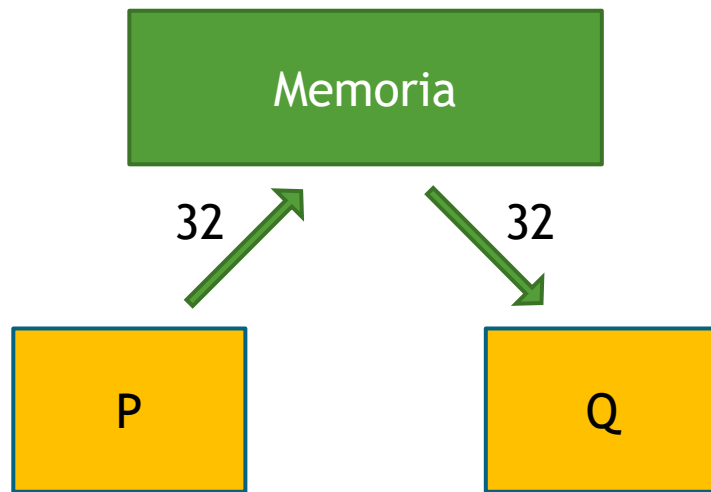


Message Passing

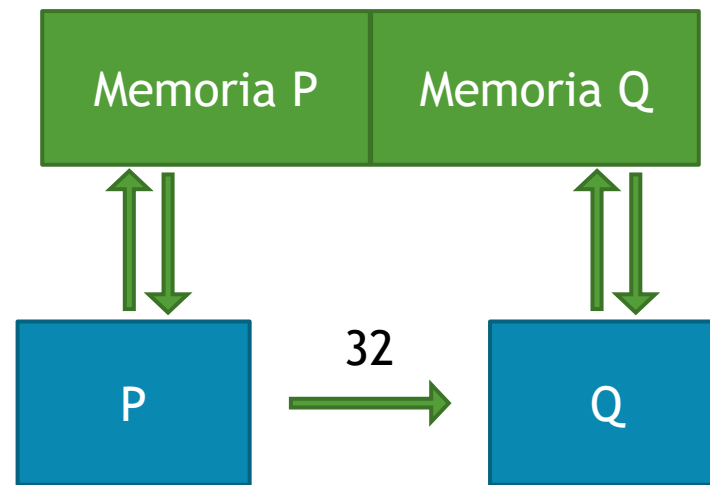
Shared Memory VS Message Passing

Message Passing (Scambio di messaggi)

- ▶ i processi accedono ad aree diverse (separate) della memoria, ma possono inviarsi messaggi e sincronizzarsi usando i servizi di **inter-process communication (IPC)** messi a disposizione dal sistema operativo



Shared Memory



Message Passing

Esempio di comunicazione con shared memory (in pseudocodice)

Modello producer-consumer: classico schema di programmazione concorrente che descrive un sistema in cui due entità principali, il **producer** e il **consumer** interagiscono attraverso una risorsa condivisa, il primo generando dati e il secondo prelevandoli

```
// variabili globali (memoria condivisa)
```

```
int x = 0;
```

```
// THREAD 1
```

```
producer() {
```

```
    int k = 6;
```

```
    x = fattoriale(k);
```

```
}
```

```
// THREAD 2
```

```
consumer() {
```

```
    while (x==0) sleep(10);
```

```
    print(x); // stampa 720
```

```
}
```

Il consumer attende (con **sleep()** l'esecuzione si sospende) che il producer abbia scritto un valore in x usando una tecnica di **busy waiting** [**attesa attiva**] (testa il valore di x ogni 10 ms)

- I due thread si **sincronizzano**: il secondo attende il primo
- la strategia **busy waiting** è **inefficiente** (il secondo thread consuma tempo di CPU solo per testare ripetutamente il valore di x)

Esempio di comunicazione con shared memory (in pseudocodice)

```
// variabili globali (memoria condivisa)
```

```
int x = 0;
```

```
// THREAD 1
```

```
producer() {
```

```
    int k = 6;
```

```
    x = fattoriale(k);
```

```
    wakeup();
```

```
}
```

```
// THREAD 2
```

```
consumer() {
```

```
    if (x==0) sleep();
```

```
    print(x); // stampa 720
```

```
}
```

Il **runtime del linguaggio** può mettere a disposizione servizi di segnalazione tra thread

- Chiamando **sleep()** l'esecuzione del thread si sospende: il consumer si mette in attesa
- Chiamando **wakeup()** il producer **sblocca** il consumer segnalandogli che il dato è pronto per essere letto

Esempio di comunicazione con message passing (in pseudocodice)

```
// nessuna memoria condivisa

// PROCESSO 1           // PROCESSO 2
producer() {             consumer() {
    int k = 6;             int y = receive();
    int x = fattoriale(k); print(y); // stampa 720
    send(x);               }
}
```

Il **SO** mette a disposizione dei processi un vero e proprio servizio di messaggistica (**Inter-Process Communication, IPC**) con primitive per mandare e ricevere messaggi

- chiamando **receive()** il processo consumer **si mette in attesa** di ricevere messaggi
- chiamando **send(x)** il processo producer **sblocca il consumer** e gli passa il valore di x
- il **trasferimento del dato** dalla memoria di un processo a quella dell'altro è a carico del SO

Necessità di meccanismi di sincronizzazione

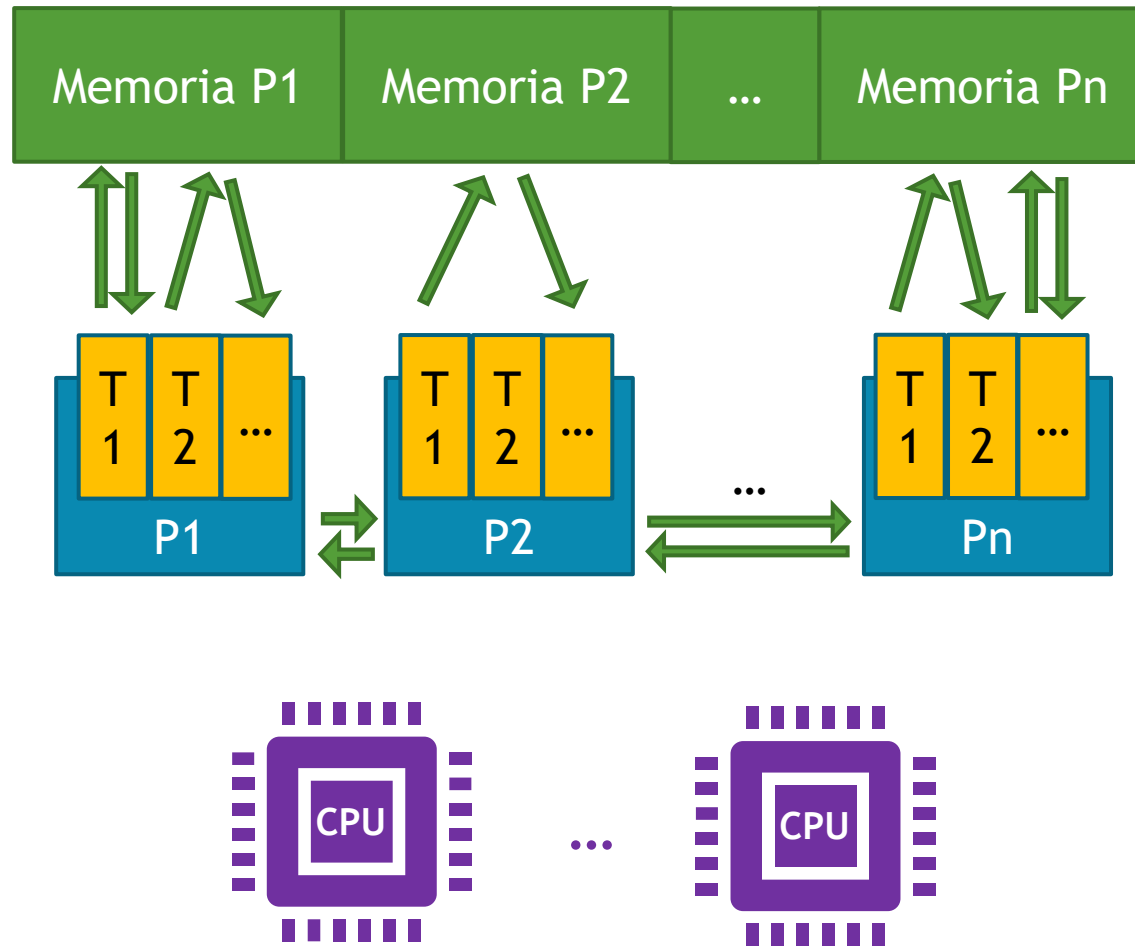
Gli esempi mostrano l'importanza dei **meccanismi di sincronizzazione**

- ▶ busy waiting
- ▶ sleep() e wakeup()
- ▶ send() e receive()
- ▶ ...

Nel caso dei **thread** questi meccanismi possono essere realizzati a livello di **runtime del linguaggio** di programmazione, dato che i thread condividono lo stesso spazio di memoria

Nel caso dei **processi**, deve necessariamente essere il **sistema operativo** a fornire gli opportuni servizi di sincronizzazione e comunicazione, dato che i processi operano in spazi di memoria separati e sono isolati tra loro

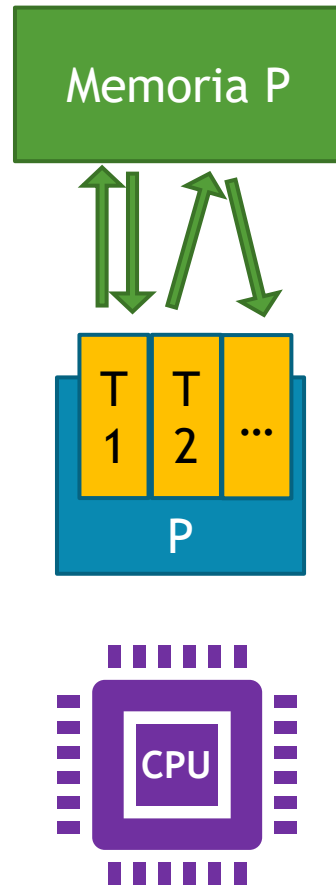
Visione astratta (e semplificata) dell'hardware e del sistema operativo



In generale:

- il sistema operativo può mantenere in esecuzione **vari processi** fornendo servizi di IPC per consentirne l'interazione
- **ogni processo** può prevedere **uno o più thread**, che condividono la stessa memoria del processo
- sia i processi sia i loro thread possono essere eseguiti **in parallelo su più CPU**

Scenario che consideriamo per studiare la concorrenza (multithreading)



Per studiare le problematiche legate alla programmazione concorrente e i meccanismi di sincronizzazione ci concentreremo solo su uno scenario **multithreading (con shared memory)** assumendo di avere un **singolo processo** e una **singola CPU**

Scenario che consideriamo per studiare la concorrenza (multithreading)

